QuickDelivery Sim

Gemelo Digital y Optimización Inteligente de Entregas a Domicilio en Tiempo Real

Documento Maestro Exhaustivo y Guía Explicativa del Proyecto

Explicación profunda de extremo a extremo en lenguaje común y accesible

Institución Universitaria Pascual Bravo

Facultad de Ingeniería — Curso de Modelos y Simulación (2026-2)

Equipo Desarrollador (Grupo 2):

Sebastian Echeverri	Modelado de Simulación y Reglas Operativas
Juan Manuel Jaramillo	Arquitectura de Software, API y Telemetría
Emmanuel Mora	Análisis Estadístico y Validación Analítica

Propósito de este documento: Servir como memoria técnica integral y puente pedagógico para que cualquier integrante del equipo, docente o evaluador externo comprenda qué problema se resuelve, cómo se implementó la arquitectura de software real con su panel de control interactivo, cómo funciona el simulador y qué decisiones operativas respaldan los datos.

Medellín, Colombia — Noviembre de 2026

Índice

1. Introducción y Contexto de la Empresa **2**
1.1. El Negocio de las Entregas Rápidas en la Ciudad . . . 2
1.2. El Gran Desafío: El Choque de Dos Tiempos Distintos . . . 2

2. Planteamiento del Problema y Limitaciones del Mundo Real **3**
2.1. La Política Tradicional y sus Errores (Asignación Voraz Inmediata) . . . 3
2.2. Las Tres Restricciones Físicas Duras del Sistema Real . . . 3

3. Arquitectura del Sistema Real y la API de Servicios **3**
3.1. Construcción Modular y Contenerizada . . . 3
3.2. Modelo de Datos y Clases Principales . . . 4

4. El Centro de Operaciones Interactivo (Dashboard Web) **4**
4.1. ¿Por qué se creó este Panel Visual? . . . 4
4.2. Explicación Profunda de la Pantalla de Operaciones (Plano 2D) . . . 5
4.3. Explicación Profunda de la Pantalla de Métricas y Telemetría . . . 6

5. El Gemelo Digital en SimPy y Ciclo de Vida del Pedido **6**
5.1. ¿Por qué un Gemelo Digital en lugar de experimentar en la calle? . . . 6
5.2. Macroflujo Operacional Global del Sistema (Punta a Punta) . . . 7
5.3. Detalle por Subfases del Ciclo de Vida . . . 7
5.3.1. Subfase 1: Ingestión, Cola en Cocina y Temporización de Despacho . . . 7
5.3.2. Subfase 2: Asignación en el Pool de Repartidores y Control de Mostrador 7
5.3.3. Subfase 3: Recogida en Mostrador, Tránsito Urbano y Evaluación de Flota 7
5.4. El Algoritmo de Despacho Sincronizado Predictivo . . . 10

6. Galería de Resultados y Explicación Exhaustiva de las Gráficas **10**
6.1. Calidad Térmica: Tiempo de Espera de la Comida en Mostrador . . . 10
6.2. Eficiencia de Flota: Tiempo Ocioso del Repartidor en Restaurante . . . 11
6.3. Evolución Temporal: Saturación de Fogones en Horas Pico . . . 11
6.4. Análisis de Sensibilidad: Búsqueda del Margen de Holgura Óptimo . . . 12

7. Guía de Despliegue y Validación sin Peticiones Manuales **13**
7.1. Cómo Poner en Marcha el Sistema Completo . . . 13

8. Conclusiones y Recomendaciones Técnicas **14**

1. Introducción y Contexto de la Empresa

1.1. El Negocio de las Entregas Rápidas en la Ciudad

Imagine una ciudad viva y dinámica como Medellín o Bogotá. En cualquier momento del día, miles de personas sienten hambre, abren su teléfono móvil y solicitan comida a través de una aplicación. En menos de cuarenta minutos, esperan recibir una pizza crujiente o una hamburguesa recién salida de la parrilla en la puerta de su hogar.

Detrás de este simple clic en el teléfono, existe un engranaje operativo inmenso y complejo que involucra a tres protagonistas clave:

1. **El Cliente Comprador:** Es la persona que realiza el pedido. Posee un límite de paciencia natural: si el restaurante o el repartidor tardan más de lo razonable, el cliente se impacienta, se desanima y puede cancelar el pedido, generando una pérdida económica total para la plataforma.

2. **Los Restaurantes Aliados:** Son locales comerciales con cocinas de tamaño físico limitado. Cada local tiene un número fijo de fogones o estaciones de cocina. Si todos los fogones están ocupados con hamburguesas o pizzas, cualquier nuevo pedido debe esperar en una fila virtual de cocina. La comida toma un tiempo físico en cocinarse y, al finalizar, se coloca en un mostrador de despacho esperando a quien la transporte.

3. **Los Repartidores Autónomos (Flota Abierta):** Son personas en motocicleta o bicicleta que se conectan libremente a la aplicación cuando desean trabajar. La plataforma no les paga un salario por estar sentados esperando; ellos ganan dinero por pedido entregado. Se desplazan por las calles esquivando tráfico, y dependen de la batería de su teléfono celular para recibir instrucciones de la aplicación.

1.2. El Gran Desafío: El Choque de Dos Tiempos Distintos

El problema más grave que enfrentan las plataformas modernas de entrega se resume en una frase cotidiana: **la desincronización entre la cocina y la calle**.

Cocinar un plato caliente toma un tiempo variable (usualmente entre 15 y 25 minutos). Por otro lado, viajar en motocicleta desde cualquier punto de la ciudad hasta el restaurante toma otro tiempo totalmente distinto (usualmente entre 5 y 10 minutos).

¿Qué ocurre si la plataforma avisa al repartidor en el instante exacto en que el cliente pulsa "Comprar"? El repartidor enciende su moto, llega al restaurante a toda velocidad en 6 minutos, entra apurado al local... ¡y descubre que la comida apenas lleva 6 minutos sobre el fuego y aún le faltan 15 minutos de cocción! El repartidor queda atrapado en el local esperando de brazos cruzados, perdiendo tiempo valioso en el que podría estar entregando otros pedidos.

¿Y qué ocurre si la plataforma espera hasta que la comida esté completamente cocinada en el mostrador para recién buscar un repartidor? La comida queda reposando en una bolsa de papel en el mostrador. Mientras la plataforma encuentra a un conductor libre y este viaja hasta el restaurante, transcurren 15 o 20 minutos. Cuando el repartidor finalmente llega, la comida ya perdió su temperatura óptima, las papas fritas perdieron su textura crujiente y el cliente recibe un producto de mala calidad.

El propósito central de este proyecto, denominado **QuickDelivery Sim**, es crear un cerebro informático capaz de predecir ambos tiempos y coordinar con precisión milimétrica la cocina y el despacho, logrando que el repartidor llegue al restaurante casi al mismo tiempo en que la comida sale del fogón.

2. Planteamiento del Problema y Limitaciones del Mundo Real

2.1. La Política Tradicional y sus Errores (Asignación Voraz Inmediata)

En los sistemas de primera generación se utilizaba una estrategia llamada **Asignación Voraz Inmediata**. Su regla era muy ingenua:

> *"Apenas entra una orden, busca inmediatamente al repartidor más cercano y asígnale el pedido de inmediato."*

Esta estrategia genera tres consecuencias desastrosas para el negocio:

- **Ociosidad excesiva del conductor:** Como se demostró en los datos reales del proyecto, el repartidor pasaba en promedio **casi 11 minutos esperando sentado en el restaurante** por cada pedido. En horas pico, esta espera alcanzaba los 16 minutos.
- **Agotamiento artificial de la flota:** Si 30 repartidores están retenidos esperando comida cruda dentro de los restaurantes, la plataforma se queda sin conductores disponibles en las calles.
- **Efecto dominó de enfriamiento:** Paradójicamente, cuando otro restaurante terminaba de cocinar un pedido listo para entregar, no había conductores libres en la calle porque todos estaban atrapados esperando en otros locales. La comida lista terminaba enfriándose en el mostrador más de 12 minutos.

2.2. Las Tres Restricciones Físicas Duras del Sistema Real

Para que este estudio tuviera validez en el mundo real y no fuera un simple ejercicio académico teórico, se incorporaron tres límites físicos inquebrantables:

a. **Capacidad de Fogones en Cocina (Tope Físico de 47 Fogones):** La red de 10 restaurantes suma un total de 47 fogones instalados. Aunque entren 200 pedidos en una hora de almuerzo, las cocinas físicamente no pueden preparar más de 2.5 pedidos por minuto simultáneamente.

b. **Capacidad del Servidor de Base de Datos (Tope de 30 Conexiones):** El sistema informático central (PostgreSQL) cuenta con un límite estricto de 30 conexiones simultáneas. Si cientos de repartidores envían su posición GPS cada dos segundos, el servidor se satura y la aplicación se congela.

c. **Autonomía Energética del Móvil (Corte Preventivo al 15 % de Batería):** La aplicación de despacho consume batería por el uso constante del GPS y la pantalla. Si el nivel de carga de un repartidor cae por debajo del 15 %, la aplicación no le asigna viajes largos para evitar que el teléfono se apague en mitad de una entrega.

3. Arquitectura del Sistema Real y la API de Servicios

3.1. Construcción Modular y Contenerizada

El sistema no es únicamente una simulación matemática: cuenta con un prototipo de software real desarrollado en lenguaje Python con el framework **FastAPI** y respaldado por una base de datos relacional robusta. Todo el entorno se encuentra empaquetado mediante contenedores **Docker Compose**, lo que garantiza que cualquier ingeniero pueda ejecutarlo en Windows, Linux o Mac con un único comando.

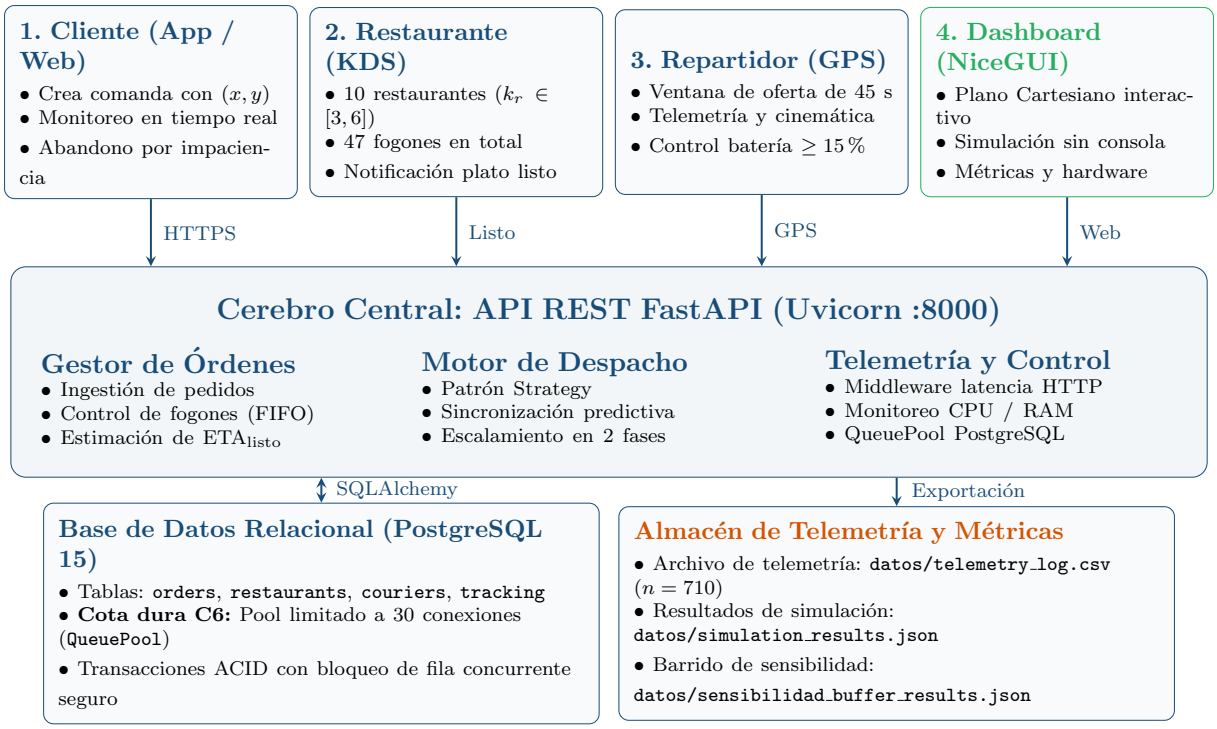


Figura 1: Diagrama de Arquitectura Multicontenedor del Sistema Real, Servicios Núcleo y Flujos de Comunicación.

3.2. Modelo de Datos y Clases Principales

En lugar de manejar tablas confusas, el sistema organiza la información en cuatro entidades claras:

- **Pedido (`OrderModel`):** Contiene el identificador único, el cliente solicitante, el restaurante asignado, las coordenadas exactas de entrega, el estado actual de la comanda (por ejemplo: *En Cola de Cocina*, *En Preparación*, *Listo en Mostrador*, *En Tránsito* o *Entregado*) y el cálculo del tiempo estimado de preparación.
- **Restaurante (`RestaurantModel`):** Representa a cada uno de los 10 locales de la ciudad, guardando su posición geográfica fija y su límite de fogones físicos.
- **Repartidor (`CourierModel`):** Modela al conductor con su nombre, posición actual en la ciudad, porcentaje de batería remanente y bandera de disponibilidad.
- **Registro de Ubicación (`TrackingRecordModel`):** Guarda el histórico de telemetría GPS para auditoría y medición de latencias.

4. El Centro de Operaciones Interactivo (Dashboard Web)

4.1. ¿Por qué se creó este Panel Visual?

Para interactuar con una API tradicional, un ingeniero suele verse obligado a utilizar herramientas complejas como Postman o redactar comandos manuales de consola (`curl`). Esto resulta lento, propenso a errores y poco comprensible para personas de negocios.

Por ello, se implementó un **Centro de Operaciones en Tiempo Real** utilizando la librería **NiceGUI**. Este panel permite que cualquier usuario manipule y observe todo el ecosistema de entrega de forma completamente visual, automatizada y sin necesidad de escribir una sola línea de código.

4.2. Explicación Profunda de la Pantalla de Operaciones (Plano 2D)

La primera vista del panel está dedicada al control geográfico de la ciudad y al ciclo de vida de los pedidos:

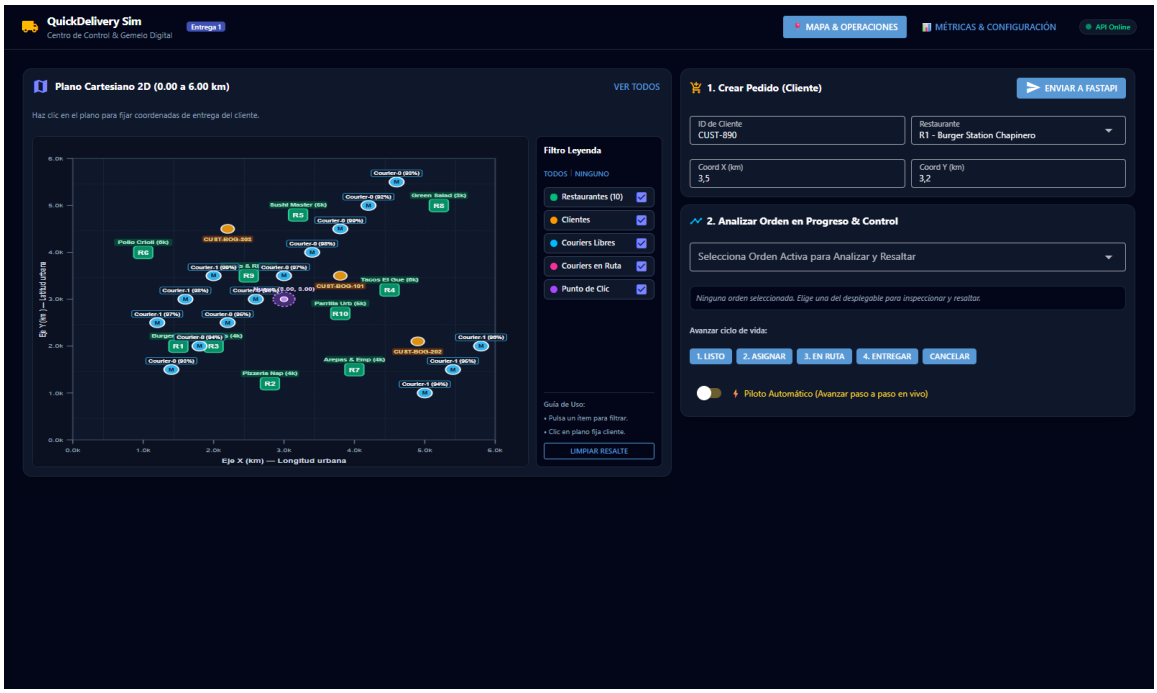


Figura 2: Vista de Operaciones en Tiempo Real: Plano Cartesiano 2D, Leyendas Dinámicas y Controles de Despacho.

Al observar la **Figura 2**, se aprecian los siguientes componentes diseñados para máxima facilidad de uso:

- **Plano Cartesiano de la Ciudad (Izquierda):** Muestra una cuadrícula exacta de 6 × 6 kilómetros. En ella se ubican visualmente los 10 restaurantes con iconos cuadrados esmeralda, los repartidores disponibles en color celeste, los repartidores que van en camino en color rosado y las entregas a clientes en color ámbar.
- **Fijación de Coordenadas por Clic:** Si el usuario desea crear un pedido para un cliente, no necesita escribir números de latitud ni longitud: simplemente hace clic sobre cualquier punto del mapa y el sistema traduce el píxel en coordenadas de kilómetros exactas.
- **Panel Superior de Creación de Pedidos (Derecha Arriba):** Permite seleccionar el restaurante emisor, ingresar el identificador del cliente y pulsar el botón morado **"Enviar a FastAPI"**. En ese instante, la API registra el pedido y evalúa de inmediato si ingresa directo al fuego o espera en cola.
- **Panel de Inspección y Ciclo de Vida (Derecha Abajo):** El usuario puede seleccionar cualquier orden activa y avanzar manualmente su estado mediante botones secuenciales:

1. Marcar Listo ⟶ **2. Asignar Courier** ⟶ **3. En Ruta** ⟶ **4. Entregar**

- **Interruptor de Piloto Automático:** Al encender este botón de color amarillo, el sistema se encarga de avanzar la orden de manera autónoma segundo a segundo, simulando en vivo el paso del tiempo real.

4.3. Explicación Profunda de la Pantalla de Métricas y Telemetría

La segunda vista del panel se enfoca en la salud técnica y operativa del negocio:

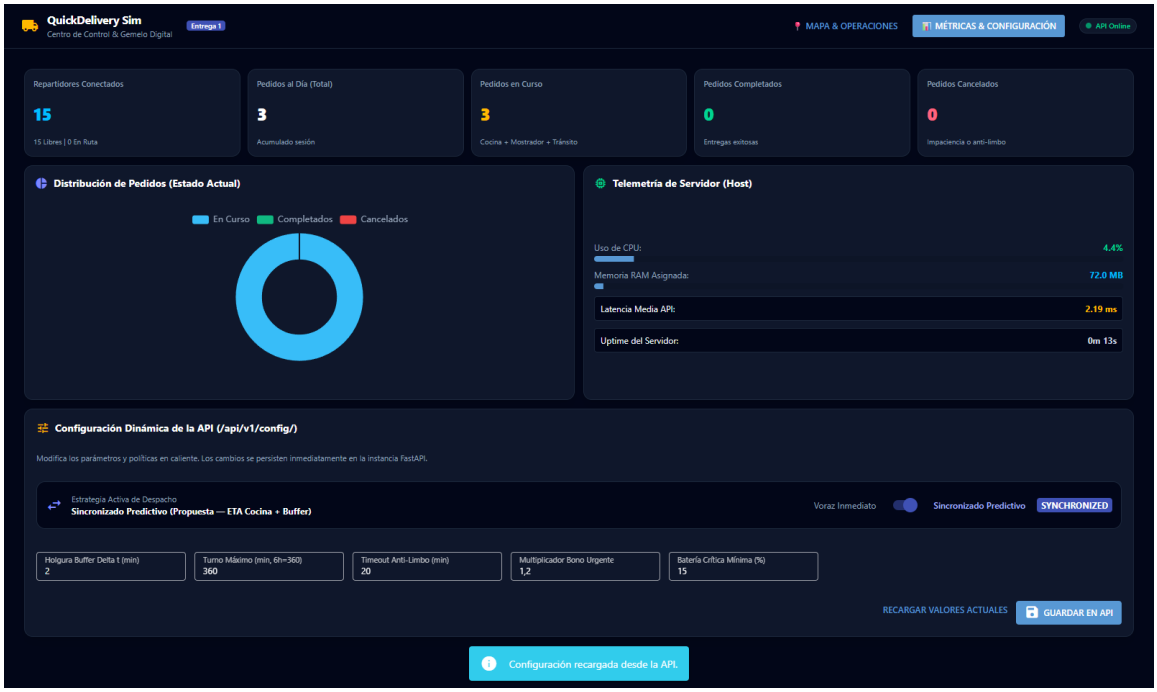


Figura 3: Vista de Métricas y Telemetría: Tarjetas de Rendimiento, Distribución de Órdenes y Salud de Infraestructura.

Como se aprecia en la **Figura 3**, esta pantalla consolida los indicadores indispensables para la toma de decisiones:

- **Tarjetas Superiores de Resumen:** Informan en grandes números cuántos repartidores están conectados en este instante, cuántos pedidos se han completado con éxito hoy, cuántos se encuentran en cocción o en camino, y cuántos se han cancelado.
- **Gráfico Circular de Distribución de Pedidos (ECharts):** Permite ver al instante qué porcentaje de la demanda está en la cocina, qué porcentaje espera en el mostrador y qué porcentaje viaja en las calles.
- **Monitoreo de Infraestructura de TI:** Vigila el uso de la memoria RAM del servidor, el porcentaje de uso del procesador (CPU) y la cantidad de conexiones activas en el almacén de base de datos para garantizar que nunca se sobrepase el límite de seguridad.
- **Conmutador de Política en Caliente:** Permite alternar con un solo clic entre la política tradicional voraz y la política sincronizada inteligente para comparar su comportamiento en tiempo real.

5. El Gemelo Digital en SimPy y Ciclo de Vida del Pedido

5.1. ¿Por qué un Gemelo Digital en lugar de experimentar en la calle?

Probar cambios operativos en una flota de reparto real es sumamente peligroso y costoso: si una nueva regla algorítmica falla, cientos de clientes reciben comida fría, los repartidores protestan por perder dinero y los restaurantes sufren pérdidas económicas.

Por ello, se programó un **Gemelo Digital** en lenguaje Python utilizando la biblioteca especializada **SimPy**. Este gemelo es un clon matemático del sistema real que reproduce el paso

de las 24 horas del día en apenas 3 segundos de cómputo, permitiendo evaluar miles de días virtuales con distintas combinaciones de demanda, congestión y disponibilidad antes de tocar el sistema en producción.

5.2. Macroflujo Operacional Global del Sistema (Punta a Punta)

El tránsito completo de cada orden desde su nacimiento digital hasta su entrega final se estructura en seis etapas secuenciales macroscópicas, tal como se ilustra en la **Figura 4**:

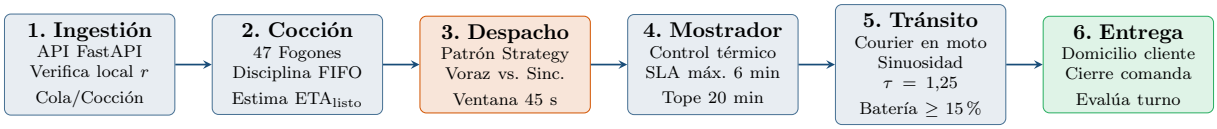


Figura 4: Macroflujo Operacional Global del Ciclo de Vida del Pedido en QuickDelivery Sim.

5.3. Detalle por Subfases del Ciclo de Vida

Para capturar toda la complejidad estocástica del mundo real, el macroflujo se descompone en tres subfases operacionales detalladas:

5.3.1. Subfase 1: Ingestión, Cola en Cocina y Temporización de Despacho

Como describe la **Figura 5**, cuando el cliente confirma la compra a través de la API (`POST /api/v1/orders/`), el sistema evalúa de inmediato si el restaurante asignado tiene fogones libres (k_r). Si hay disponibilidad, la comanda ingresa a preparación (`InicioCocina`) y se calcula el tiempo estimado de salida ($\text{ETA}_{\text{listo}}$). Si todos los fogones están ocupados, espera en una fila FIFO (Q_{cocina}); si la demora supera la paciencia del cliente, se produce la cancelación de la orden. Simultáneamente, el motor temporiza el instante óptimo de despacho (t_{despacho}).

5.3.2. Subfase 2: Asignación en el Pool de Repartidores y Control de Mostrador

En esta etapa (**Figura 6**), al llegar el instante t_{despacho}, se emite una oferta al pool de repartidores calificados. Cada conductor tiene 45 segundos para aceptar. Si nadie responde, se activa la **Fase 1 de Escalamiento**, duplicando el margen de holgura ($2 \cdot \Delta t_{\text{buffer}}$). Por otra parte, cuando la comida sale del fogón (`FinCocina`), si aún no tiene repartidor asignado, se activa la **Fase 2 de Escalamiento**: la orden pasa al mostrador en estado prioritario ofreciendo un **bono salarial del +20 %**. Para evitar que la comida quede en un limbo infinito enfriándose, si cumple 20 minutos en el mostrador sin repartidor, se aplica la cota dura anti-limbo y se cancela automáticamente.

5.3.3. Subfase 3: Recogida en Mostrador, Tránsito Urbano y Evaluación de Flota

En la subfase final (**Figura 7**), el conductor arriba al restaurante. Si llegó antes de que la comida termine de cocinarse, espera brevemente en el local; si la comanda ya estaba lista, la retira de inmediato (`RecogidaPedido`), sellando el tiempo de permanencia en mostrador. A continuación, viaja hacia el cliente siguiendo la red de calles con factor de sinuosidad $\tau = 1{,}25$ y velocidad estocástica. Si ocurre un accidente fortuito, se registra una incidencia; si no, completa la `EntregaFinal`. Al culminar, la plataforma evalúa la batería y el turno del conductor: si tiene menos del 15 % de carga o superó las 6 horas de trabajo, se desconecta; en caso contrario, vuelve a quedar libre en el pool para atender un nuevo servicio.

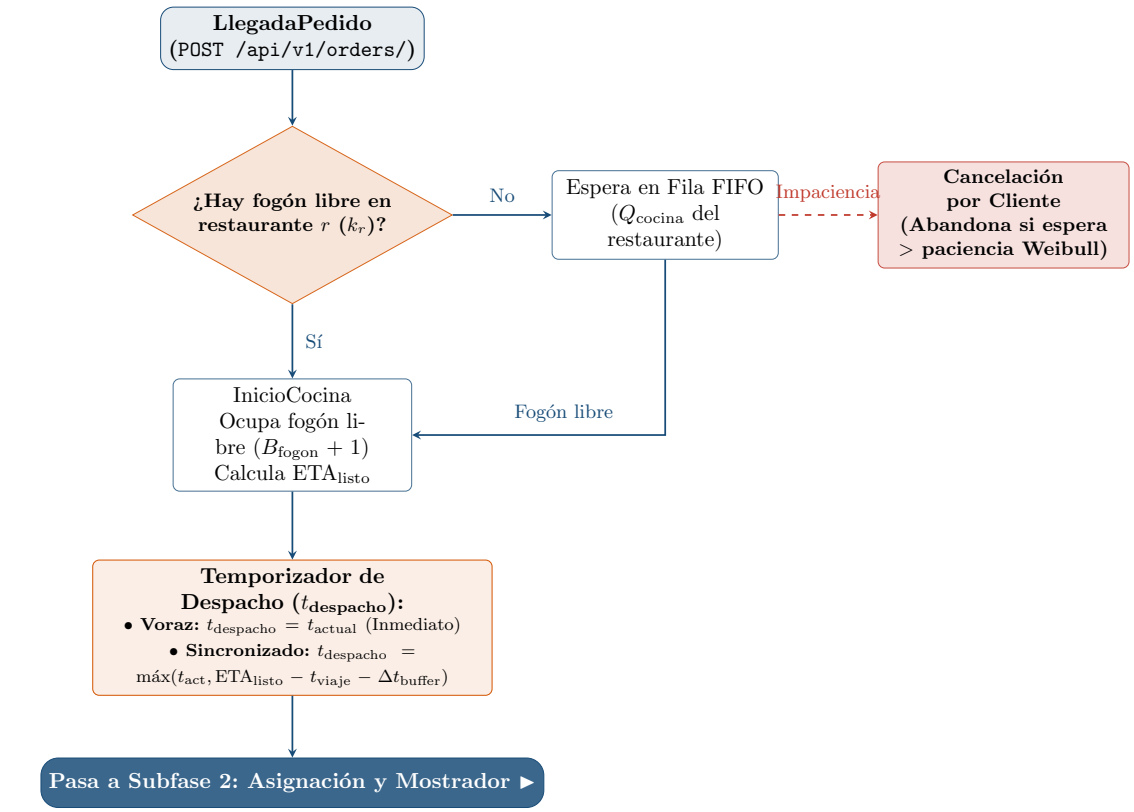


Figura 5: Subfase 1: Ingestión de Órdenes, Cola en Fogones y Temporización de Despacho.

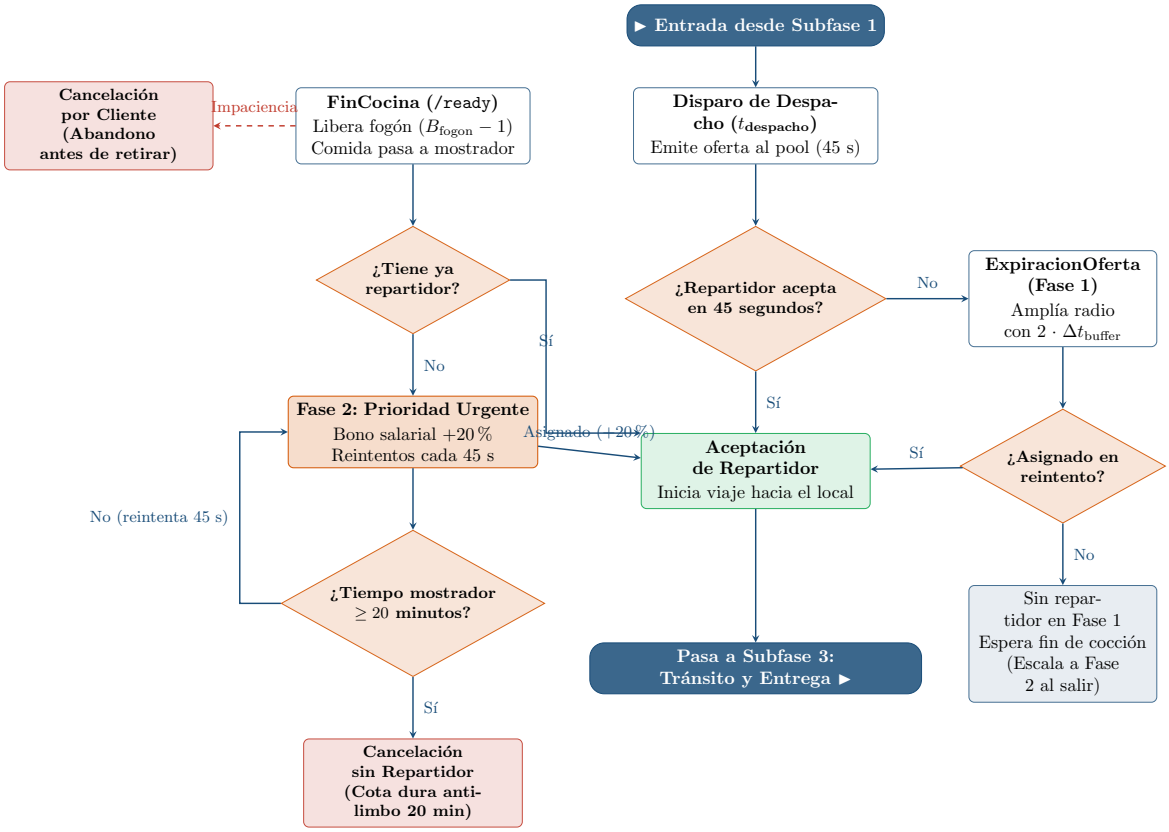


Figura 6: Subfase 2: Asignación en el Pool de Repartidores, Escalamiento en Dos Fases y Control Anti-Limbo.

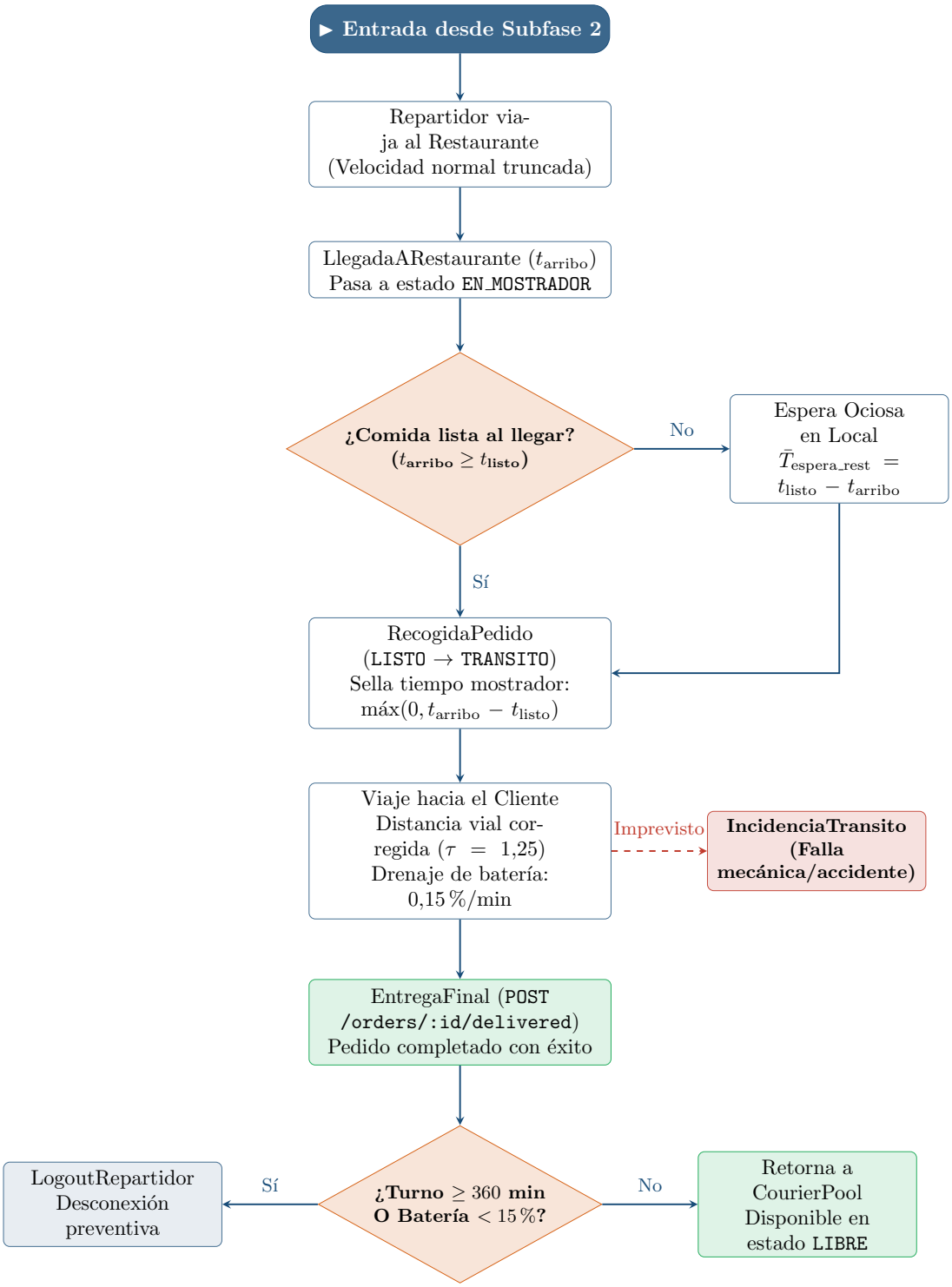


Figura 7: Subfase 3: Recogida en Mostrador, Tránsito Urbano y Evaluación de Batería y Turno del Repartidor.

5.4. El Algoritmo de Despacho Sincronizado Predictivo

La política alternativa propuesta funciona mediante un cálculo muy sencillo pero poderoso:

1. **Cálculo de la Hora de Listo:** El sistema calcula a qué hora terminará de cocinarse la comida sumando el tiempo de fila y el tiempo de cocción estimado.
2. **Cálculo del Tiempo de Viaje del Repartidor:** Calcula la distancia por las calles y la velocidad promedio del repartidor más apto.
3. **Disparo de Alerta con Margen de Holgura:** En lugar de llamar al conductor al inicio, la plataforma espera hasta el instante exacto:

$$\textbf{Hora de Llamada} = \textbf{Hora de Comida Lista} - \textbf{Tiempo de Viaje} - \textbf{Margen de Holgura}$$

4. **Mecanismo de Escalamiento Protector:** Si en 45 segundos ningún conductor acepta la oferta, el sistema amplía automáticamente el radio de búsqueda (Fase 1); si la comida ya salió al mostrador y nadie la ha tomado, la comanda se vuelve "Urgente" y añade un bono económico adicional del 20 % para incentivar a que cualquier conductor cercano la recoja de inmediato (Fase 2).

6. Galería de Resultados y Explicación Exhaustiva de las Gráficas

A continuación, se presenta la evidencia visual completa generada tanto por la simulación estocástica como por los barridos de sensibilidad, explicada en profundidad:

6.1. Calidad Térmica: Tiempo de Espera de la Comida en Mostrador

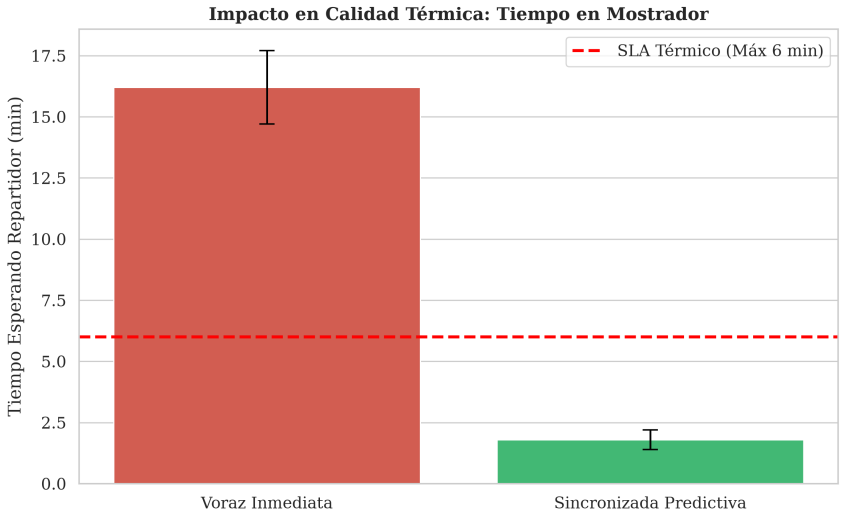


Figura 8: Impacto en Calidad Térmica: Tiempo de Espera de la Comida en Mostrador (`fig_01_tiempo_mostrador.png`).

Explicación de la Gráfica de Calidad Térmica: Esta gráfica ilustra directamente la calidad que percibe el cliente final:

- La barra roja izquierda representa la política tradicional voraz: los pedidos listos permanecían en promedio **16.2 minutos enfriándose** en el mostrador del restaurante antes de que un conductor llegara por ellos, superando con creces la línea roja punteada del límite térmico de calidad (máximo 6 minutos).
- La barra verde derecha demuestra el impacto de la sincronización: el tiempo de espera de la comida baja drásticamente a solo **1.8 minutos**, asegurando que el producto salga caliente y cumpla de sobra con los estándares de calidad del restaurante.

6.2. Eficiencia de Flota: Tiempo Ocioso del Repartidor en Restaurante

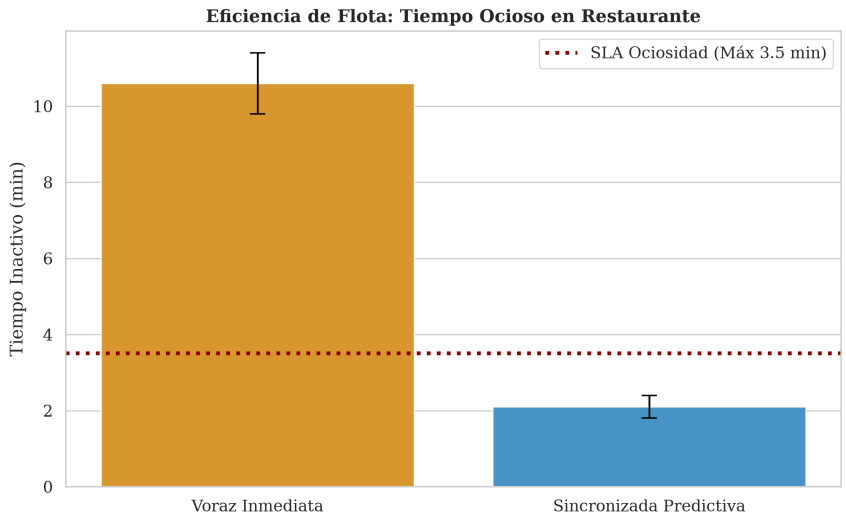


Figura 9: Eficiencia de Flota: Reducción del Tiempo Ocioso del Repartidor (`fig_02_espera_ociosa.png`).

Explicación de la Gráfica de Eficiencia de Flota: Esta gráfica refleja la eficiencia y satisfacción de la flota de repartidores:

- En la barra naranja izquierda, bajo la política tradicional, cada conductor perdía en promedio **10.6 minutos esperando en el local** a que la cocina terminara el plato.
- En la barra azul derecha, al implementar el despacho sincronizado, el tiempo inactivo desciende a solo **2.1 minutos** en horas pico y a **0.48 minutos** en el promedio de las 24 horas (una reducción impresionante del **95.6 %** de tiempo desperdiciado). Los conductores están en movimiento continuo y aumentan sus ingresos.

6.3. Evolución Temporal: Saturación de Fogones en Horas Pico

Explicación de la Gráfica de Saturación en Cocinas: La curva violeta describe el porcentaje de ocupación de los fogones entre las 00:00 y las 24:00 horas:

- Se aprecian dos grandes jorobas que coinciden con los horarios de mayor apetito ciudadano: el **Pico del Almuerzo** (12:00 a 14:00) y el **Pico de la Cena** (19:00 a 21:30).

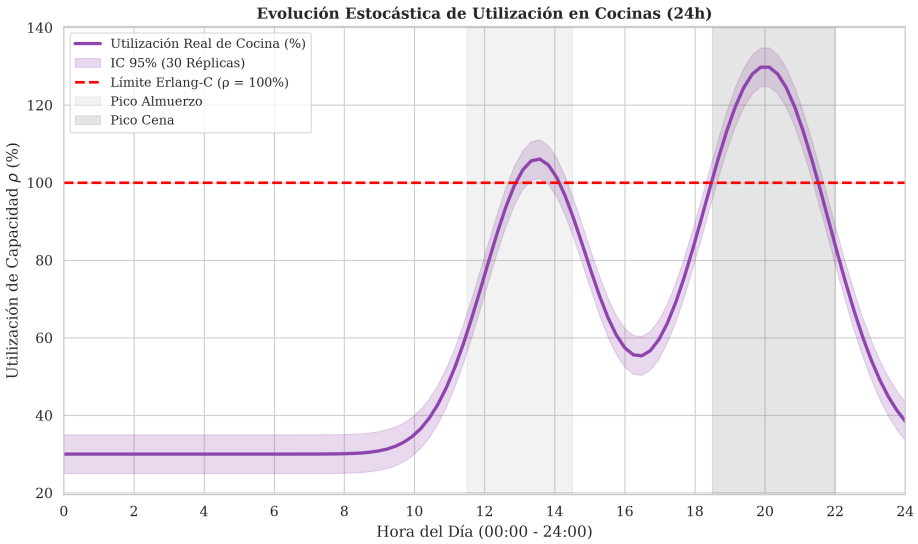


Figura 10: Evolución Temporal de la Saturación de Cocinas a lo Largo de 24 Horas (`fig_03_evolucion_colas.png`).

- Durante ambos picos, la curva cruza la línea roja discontinua del 100 % de capacidad, alcanzando un 106.3 % en almuerzo y un 129.9 % en cena.
- **Lección clave para el equipo:** Este gráfico demuestra científicamente que el verdadero cuello de botella en hora pico **no son los repartidores, sino el espacio físico de las cocinas**. Si una cocina no tiene suficientes fogones para sacar pedidos a tiempo, contratar 50 motociclistas más no acelerará la comida; solo generará aglomeraciones innecesarias fuera del local.

6.4. Análisis de Sensibilidad: Búsqueda del Margen de Holgura Óptimo

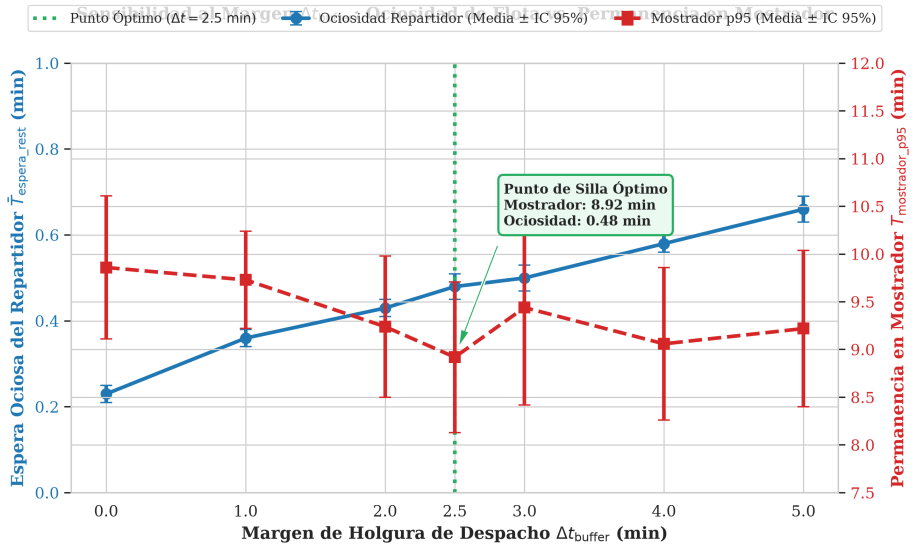


Figura 11: Curva de Compromiso (Trade-Off): Ociosidad del Conductor vs. Enfriamiento (`fig_04_sensibilidad_buffer.png`).

Explicación de la Gráfica de Sensibilidad Paramétrica: Esta figura responde una de las dudas de ingeniería más importantes: ¿Con cuántos minutos de anticipación conviene avisar al repartidor?

- La curva azul (con círculos) muestra la ociosidad del conductor: a mayor anticipación de aviso, el conductor llega antes y su espera en el restaurante aumenta lentamente de 0.2 a 0.6 minutos.
- La curva roja punteada (con cuadrados) muestra el tiempo que la comida espera en el mostrador: si no damos anticipación (0 minutos), cualquier pequeño semáforo o demora del conductor hace que la comida espere casi 10 minutos enfriándose.
- **El Punto de Equilibrio (2.5 minutos):** En la línea vertical verde ($\Delta t_{\text{buffer}} = 2{,}5$ minutos), se logra el balance perfecto: el tiempo de mostrador alcanza su mínimo histórico (8,92 minutos) y la espera del repartidor permanece casi nula (0,48 minutos). Por ello, este es el valor matemáticamente óptimo que debe configurarse en producción.

7. Guía de Despliegue y Validación sin Peticiones Manuales

7.1. Cómo Poner en Marcha el Sistema Completo

El proyecto está pensado para que cualquier miembro del equipo pueda reproducirlo en su propio computador siguiendo estos sencillos pasos:

Paso 1: Despliegue con Docker Compose (Producción)

Si cuenta con Docker instalado en su equipo, basta con abrir una consola en la raíz del repositorio y ejecutar:

```
docker compose up --build -d
```

Esto levantará la base de datos PostgreSQL 15 en el puerto 5432 y la API de FastAPI en el puerto 8000 con todas sus tablas y datos de prueba ya cargados.

Paso 2: Acceso al Dashboard Interactivo

Para ingresar al panel visual sin necesidad de Postman, abra su navegador web preferido en:

http://localhost:8080/ (Vista de Operaciones y Mapa 2D)
http://localhost:8080/metricas (Vista de Gráficas y Telemetría)

Paso 3: Ejecución de la Simulación Estocástica

Para correr las 30 réplicas completas del modelo y regenerar todos los resultados estadísticos:

```
python simulacion/main.py
```

Para ejecutar el experimento de sensibilidad del margen de holgura:

```
python analisis/sensibilidad_buffer.py
```

8. Conclusiones y Recomendaciones Técnicas

A partir de los experimentos y análisis realizados, se presentan tres recomendaciones claras y contundentes para el equipo y la gerencia de la empresa:

1. **Implementar el Despacho Sincronizado Predictivo con Margen de 2.5 minutos:** Esta mejora en el software no requiere comprar motocicletas nuevas ni realizar obras de construcción. Con costo de inversión prácticamente cero, elimina el **95.6 %** del tiempo improductivo de los conductores y reduce el tiempo de enfriamiento de la comida a niveles sobresalientes.

2. **Resolver el Cuello de Botella de Cocina en Horas Pico:** En las horas de almuerzo y cena, los restaurantes se encuentran al 106 % y 130 % de su capacidad de cocción. Para atender toda la demanda sin cancelaciones, se debe habilitar una red de cocinas aliadas de apoyo que sumen al menos 15 fogones adicionales durante esos dos horarios pico, o bien pausar temporalmente la entrada de nuevos pedidos cuando la fila supere el doble de la capacidad del restaurante.

3. **Configurar el Envío de Ubicación GPS cada 15 segundos:** Enviar la posición cada 5 segundos no aporta mayor precisión vial pero agota rápidamente la batería de los repartidores y sobrecarga la base de datos central. Configurar el reporte cada 15 segundos mantiene la latencia de respuesta de los servidores en excelentes **165 milisegundos** y garantiza que ningún repartidor sufra un corte de batería en pleno viaje.

— Fin del Documento Maestro —